

PHP - Fundamentos Teóricos

Sintaxe Básica e Principais Operadores

O PHP (Hypertext Preprocessor) é uma linguagem de script interpretada, executada no lado do servidor. Todo código PHP é delimitado por tags especiais que informam ao interpretador onde o processamento deve iniciar e finalizar.

A linguagem é **case-insensitive** para funções e palavras reservadas, mas **case-sensitive** para nomes de variáveis. Cada instrução deve ser finalizada com ponto e vírgula, exceto a última instrução antes do fechamento da tag.

Categorias de operadores:

- **Aritméticos:** realizam cálculos matemáticos tradicionais (adição, subtração, multiplicação, divisão, módulo e exponenciação)
 - **Atribuição:** o operador básico atribui valor, enquanto os compostos combinam operação aritmética com atribuição em uma única expressão
 - **Comparação:** avaliam relações entre valores, retornando booleanos. O PHP distingue comparação de valor (loose) e comparação de valor e tipo (strict)
 - **Lógicos:** combinam expressões booleanas através de conjunção, disjunção e negação
 - **Incremento/Decremento:** modificam valores numéricos em uma unidade, podendo ser aplicados antes ou depois da avaliação da expressão
 - **String:** o operador de concatenação une textos, existindo também sua versão combinada com atribuição
-

Tipos de Dados

O PHP é uma linguagem **dinamicamente tipada**, significando que o tipo de uma variável é determinado em tempo de execução conforme o valor atribuído. A partir do PHP 7+, é possível declarar tipos explicitamente, tornando o código mais robusto.

Tipos escalares (armazenam um único valor):

- **Integer:** números inteiros, positivos ou negativos, sem parte decimal. O tamanho depende da plataforma (geralmente 32 ou 64 bits)
- **Float** (ou double): números com ponto flutuante. Importante: comparações diretas entre floats podem ser imprecisas devido à representação binária
- **String:** sequências de caracteres. O PHP não possui tipo nativo para caractere individual - um caractere é simplesmente uma string de tamanho um

- **Boolean:** representa verdade lógica, aceitando apenas dois estados. Diversos valores são convertidos implicitamente para false (zero, string vazia, array vazio, null)

Tipos compostos (armazenam múltiplos valores):

- **Array:** estrutura ordenada que mapeia chaves (inteiras ou strings) a valores de qualquer tipo. Arrays PHP são na verdade *ordered maps*, extremamente versáteis
- **Object:** instância de uma classe, encapsulando dados e comportamentos relacionados

Tipos especiais:

- **NULL:** representa ausência de valor. Uma variável é null quando foi explicitamente atribuída, não foi inicializada, ou foi destruída com unset
- **Resource:** referência a recurso externo (conexão de banco, arquivo aberto, etc.)

Type Juggling: o PHP converte tipos automaticamente conforme o contexto, o que pode gerar comportamentos inesperados se não compreendido adequadamente.

Definição e Manipulação de Variáveis

Variáveis em PHP são identificadores que armazenam valores na memória. Todas são precedidas pelo símbolo cifrão, seguido de um nome que deve iniciar com letra ou underscore (nunca número).

Características fundamentais:

- Não exigem declaração prévia - são criadas no momento da primeira atribuição
- Possuem escopo que determina sua visibilidade (global, local, ou estático)
- São referências a valores na memória, podendo múltiplas variáveis apontar para o mesmo dado

Escopo de variáveis:

- **Local:** declaradas dentro de funções, existem apenas durante a execução da função
- **Global:** declaradas fora de funções, não são automaticamente acessíveis dentro delas (diferente de outras linguagens)
- **Estático:** variáveis locais que preservam seu valor entre chamadas da função
- **Superglobais:** arrays predefinidos acessíveis em qualquer escopo, contendo informações do ambiente, requisição, sessão, etc.

Constantes: valores imutáveis definidos uma única vez durante a execução. Diferem de variáveis por não usarem cifrão e não poderem ser redefinidas. Constantes são sempre globais.

Variáveis variáveis: recurso onde o nome de uma variável é determinado dinamicamente pelo valor de outra variável - poderoso mas deve ser usado com cautela por questões de legibilidade.

Estruturas de Controle

Estruturas de controle direcionam o fluxo de execução do programa baseando-se em condições avaliadas em tempo de execução. Permitem que o código tome decisões.

if/else/elseif: estrutura condicional fundamental. Avalia uma expressão booleana e executa blocos diferentes conforme o resultado. O `elseif` permite encadear múltiplas condições mutuamente exclusivas de forma legível.

switch: alternativa ao encadeamento de múltiplos `elseif` quando se compara uma única variável contra diversos valores possíveis. Utiliza comparação loose (`==`). O fluxo continua executando cases subsequentes até encontrar um `break`, característica chamada *fall-through* que pode ser intencional ou fonte de bugs.

match (PHP 8+): versão moderna e mais segura do `switch`. Utiliza comparação strict (`===`), é uma expressão (retorna valor), não possui *fall-through*, e lança exceção se nenhum caso corresponder.

Operador ternário: forma condensada do `if/else` para atribuições simples. Existe também o *null coalescing operator* para tratamento específico de valores null.

Boas práticas: evitar aninhamento excessivo (mais de 3 níveis indica necessidade de refatoração), preferir retornos antecipados para reduzir complexidade, e usar `match` quando apropriado por sua segurança de tipos.

Estruturas de Repetição

Estruturas de repetição (loops) executam um bloco de código múltiplas vezes enquanto uma condição permanece verdadeira, fundamentais para processamento de coleções e tarefas iterativas.

while: avalia a condição antes de cada iteração. Se inicialmente falsa, o bloco nunca executa. Ideal quando o número de iterações é desconhecido e depende de condição dinâmica.

do-while: avalia a condição após cada iteração, garantindo pelo menos uma execução do bloco. Útil quando a primeira execução é sempre necessária (validação de entrada, menus interativos).

for: estrutura com três expressões (inicialização, condição, incremento) separadas por ponto e vírgula. Todas são opcionais. Ideal quando o número de iterações é conhecido antecipadamente.

foreach: projetado especificamente para iteração sobre arrays e objetos iteráveis. Mais legível e menos propenso a erros que for tradicional para este propósito. Pode acessar apenas valores ou pares chave-valor.

Controle de fluxo dentro de loops:

- **break:** interrompe completamente o loop, transferindo execução para após seu bloco. Aceita argumento numérico para sair de múltiplos loops aninhados
- **continue:** pula o restante da iteração atual e avança para a próxima. Também aceita argumento numérico

Loops infinitos: ocorrem quando a condição de saída nunca é atingida - podem ser intencionais (servidores, daemons) ou bugs. Sempre garantir que existe caminho para a condição tornar-se falsa.

Performance: para grandes volumes de dados, a forma como o loop é estruturado impacta significativamente. Evitar chamadas de função na condição de verificação, preferir incrementos simples, e considerar estruturas de dados apropriadas.